3D WEBSOFT INC. FOUNDATION

FUTURE TECH SCHOOL PROGRAM

Faculty Name :

Course Duration : 5 Months

Classes : Offline & Online

Daily Teaching : 2 Hours

Total Classes : 100

Level 4 – AI Professional Foundation(11th–12th Standard)

Class 21 – Inheritance & Polymorphism



```python
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

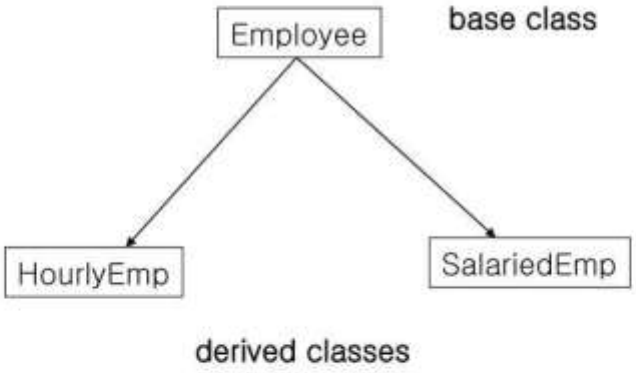
class Employee(Person):
    def __init__(self, name, age, employee_id):
        super().__init__(name, age)
        self.employee_id = employee_id

class Manager(Employee):
    def manage_team(self):
        return "Managing the team"

class Developer(Employee):
    def write_code(self):
        return "Writing code"
```

FOUNDATION

Employee Inheritance Hierarchy




```python
class Employee:
    def calculateSalary(self):
        pass

class FullTimeEmployee(Employee):
    def calculateSalary(self):
        return "Full-time Employee Salary: 5000"

class PartTimeEmployee(Employee):
    def calculateSalary(self):
        return "Part-time Employee Salary: 2000"

class Intern(Employee):
    def calculateSalary(self):
        return "Intern Salary: 1000"

# Using polymorphism
employees = [FullTimeEmployee(), PartTimeEmployee(), Intern()]

for employee in employees:
    print(f"{employee.__class__.__name__}: {employee.calculateSalary()}")
```

1. Learning Objectives

By the end of this lesson, students will be able to:

- Understand **Inheritance** in Python.
- Understand **Parent and Child classes**.
- Explain why inheritance is useful.
- Create single and multilevel inheritance.
- Use super() to access parent-class functionality.
- Understand **Polymorphism**.
- Implement **method overriding**.
- Build an Employee Management example.
- Apply inheritance and polymorphism to a real-world application.

3DWEBSOFT FOUNDATION

2. What is Inheritance?

Inheritance is an OOP feature where one class gets the properties and methods of another class.

The existing class is called the:

Parent / Base / Superclass

The new class is called the:

Child / Derived / Subclass

Real-world example

Suppose we have a general Employee class.

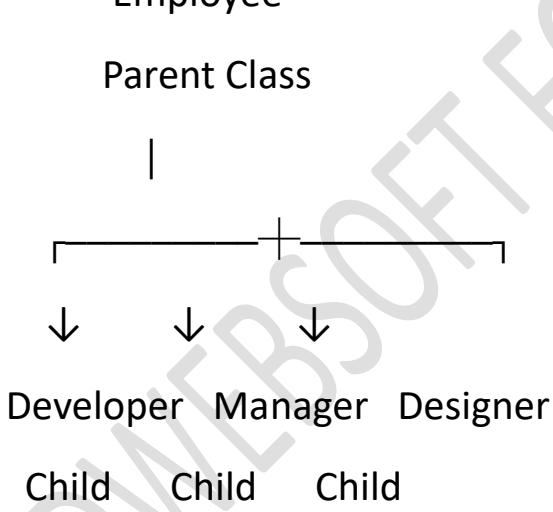
All employees may have:

- Name
- Employee ID
- Salary
- Display details

But different employees can have different roles:

- Developer
- Manager
- Designer

Instead of creating everything from scratch, we can inherit common features from Employee.



3. Why Do We Need Inheritance?

Without inheritance, we may repeat the same code.

For example:

Developer → name, ID, salary

Manager → name, ID, salary

3DWEBSOFT FOUNDATION

Designer → name, ID, salary

The common code can instead be placed in Employee.

Employee

├— name

├— employee_id

├— salary

└— display()

↓ inheritance

Developer

Manager

Designer

Main benefits

1. **Code Reusability**
2. **Less Code Duplication**
3. **Easy Maintenance**
4. **Better Organization**
5. **Supports Polymorphism**

4. Basic Inheritance Syntax

class Parent:

parent class

3DWEBSOFT FOUNDATION

```
    pass
class Child(Parent):
    # child class
    pass
```

The important part is:

```
class Child(Parent):
```

It means:

Child inherits from Parent.

5. Simple Example

```
class Employee:
    def display(self):
        print("Employee details")
class Developer(Employee):
    def code(self):
        print("Developer writes code")
d1 = Developer()
d1.display()
d1.code()
```

Output

```
Employee details
Developer writes code
```

The Developer object can use:

SUWEBSOFT FOUNDATION

display()

because it inherited the method from Employee.

6. Parent and Child Class

Parent Class

Employee

|

inheritance

↓

Child Class

Developer

Parent

class Employee:

Child

class Developer(Employee):

7. Employee Management Example

Let's create a basic employee class.

class Employee:

```
    def __init__(self, name, emp_id, salary):
        self.name = name
        self.emp_id = emp_id
```

3DWEBSOFT FOUNDATION

```
  self.salary = salary
   def display(self):
      print("Name:", self.name)
      print("Employee ID:", self.emp_id)
      print("Salary:", self.salary)
```

Now create a Developer class.

```
class Developer(Employee):
   def coding(self):
      print(self.name, "is writing Python code")
```

Create an object:

```
d1 = Developer("Rahul", 101, 50000)


d1.display()
d1.coding()
```

Output

```
Name: Rahul
Employee ID: 101
Salary: 50000
Rahul is writing Python code
```

8. Using super()

super() is used to access the parent class.

Suppose the child class has its own constructor.

JUWEBSOFT FOUNDATION

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
class Developer(Employee):
    def __init__(self, name, salary, language):
        super().__init__(name, salary)
        self.language = language
    def display(self):
        print("Name:", self.name)
        print("Salary:", self.salary)
        print("Language:", self.language)
d1 = Developer("Asha", 60000, "Python")
d1.display()
```

Output

Name: Asha

Salary: 60000

Language: Python

Important

super().__init__(name, salary)

calls the constructor of the parent class.

9. What is Polymorphism?

SUNWEBSOFT FOUNDATION

Polymorphism means:

One interface, many forms.

The same method name can behave differently for different objects.

For example:

Employee

```
  |
  ├── Developer → work() → Writes code
  |
  ├── Manager   → work() → Manages team
  |
  └── Designer  → work() → Designs graphics
```

All classes use:

work()

but each class gives the method a different behavior.

3DWEBSOFT FOUNDATION

10. Method Overriding

Method overriding occurs when a child class provides its own version of a method already defined in the parent class.

Example:

```
class Employee:
    def work(self):
        print("Employee is working")
class Developer(Employee):
```

```
    def work(self):
        print("Developer is writing code")
class Manager(Employee):
    def work(self):
        print("Manager is managing the team")
```

Now:

```
d1 = Developer()
m1 = Manager()
d1.work()
m1.work()
```

Output

```
Developer is writing code
Manager is managing the team
```

The same method:

```
work()
```

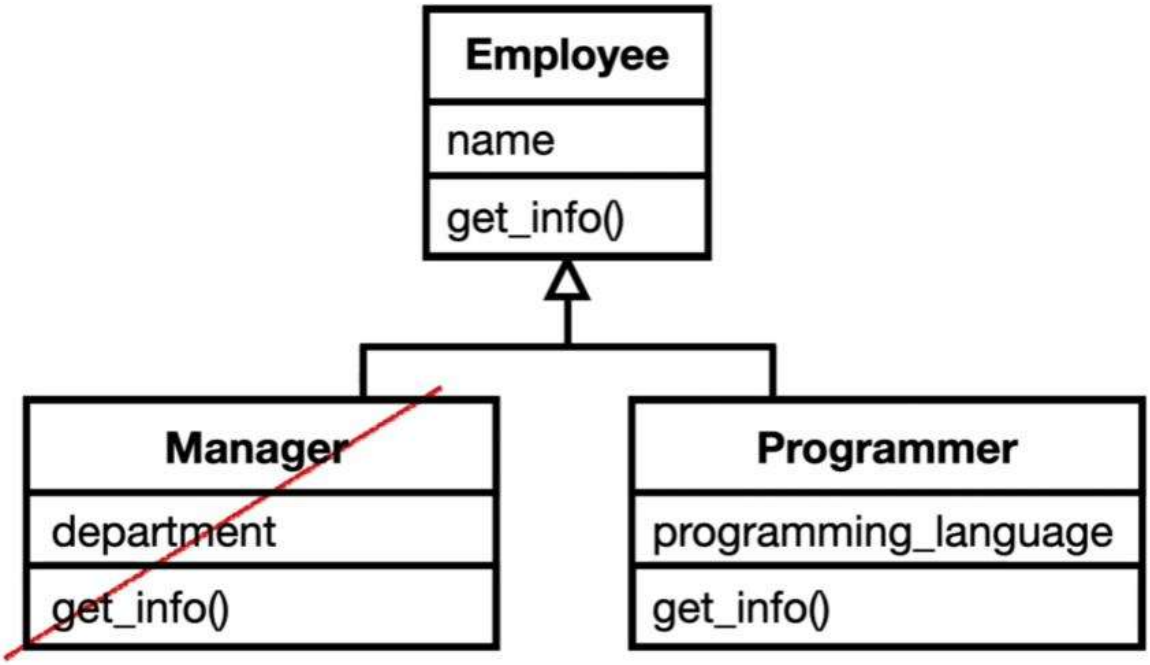
has different behavior.

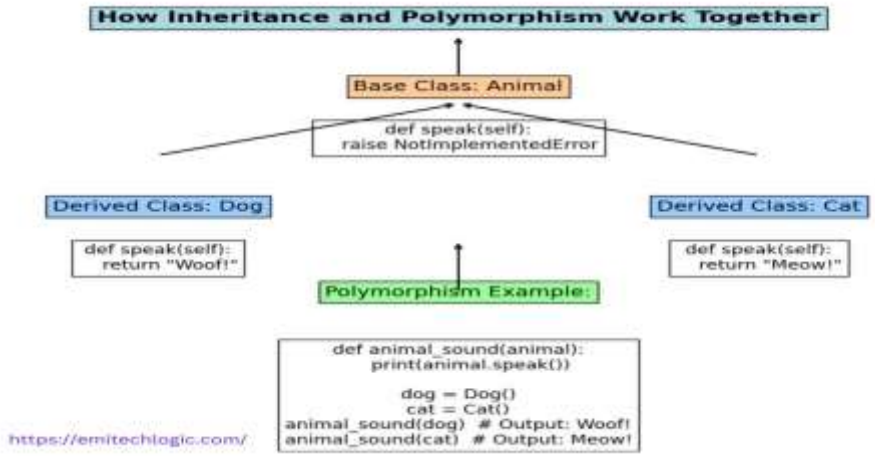
That is **polymorphism**.

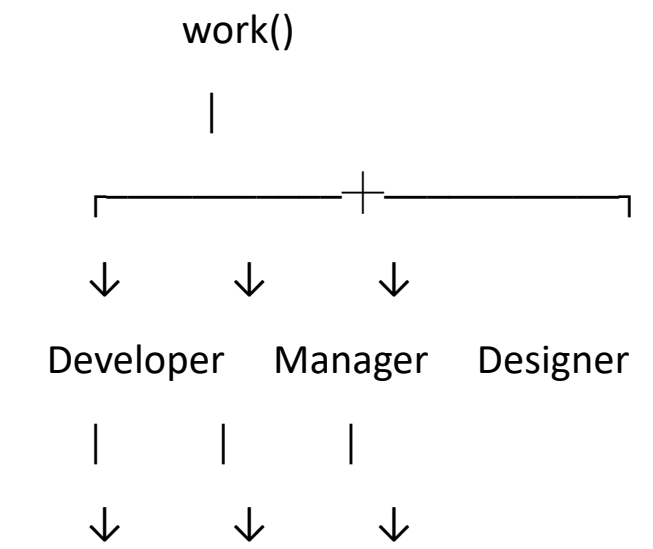
3DWEBSOFT FOUNDATION

11. Visualizing Polymorphism


Python OOP Series • Article 5
Polymorphism in Python
Same Method, Different Behavior
speak()
bark()
meow()
speak()
Python OOP Series – Article 5


Employee
name
get_info()
Manager
department
get_info()
Programmer
programming_language
get_info()


How Inheritance and Polymorphism Work Together
Base Class: Animal
def speak(self):
raise NotImplementedError
Derived Class: Dog
def speak(self):
return "Woof!"
Derived Class: Cat
def speak(self):
return "Meow!"
Polymorphism Example:
def animal_sound(animal):
print(animal.speak())
dog = Dog()
cat = Cat()
animal_sound(dog) # Output: Woof!
animal_sound(cat) # Output: Meow!
https://emitechlogic.com/



Same method:

work()

Different behavior.

12. Employee Management with Polymorphism

Let's build a practical example.

```
class Employee:

    def __init__(self, name, emp_id):
        self.name = name
        self.emp_id = emp_id
    def work(self):
        print(self.name, "is working")
class Developer(Employee):
    def work(self):
```

3DWEBSOFT FOUNDATION

```
        print(self.name, "is developing software")
class Manager(Employee):
    def work(self):
        print(self.name, "is managing the team")

class Designer(Employee):
    def work(self):
        print(self.name, "is designing graphics")

e1 = Developer("Rahul", 101)
e2 = Manager("Asha", 102)
e3 = Designer("Ravi", 103)
e1.work()
e2.work()
e3.work()
```

Output

```
Rahul is developing software
Asha is managing the team
Ravi is designing graphics
```

13. Understanding the Program

Step 1

Create the parent class:

3DWEBSOFT FOUNDATION

class Employee:

Step 2

Create child classes:

class Developer(Employee):

class Manager(Employee):

class Designer(Employee):

Step 3

Override the work() method.

Step 4

Create objects:

e1 = Developer(...)

e2 = Manager(...)

e3 = Designer(...)

Step 5

Call the same method:

e1.work()

e2.work()

e3.work()

Python chooses the appropriate version of work() based on the object.

14. Polymorphism with a Loop

3JWEBSOFT FOUNDATION

One of the most useful features of polymorphism is that different objects can be handled using the same code.

```
class Employee:


    def work(self):
        print("Employee is working")
class Developer(Employee):
    def work(self):
        print("Developer writes code")
class Manager(Employee):
    def work(self):
        print("Manager manages employees")
class Designer(Employee):
    def work(self):
        print("Designer creates designs")
employees = [
    Developer(),
    Manager(),
    Designer()
]


for employee in employees:
    employee.work()
```

3DWEBSOFT FOUNDATION

Output

Developer writes code

Manager manages employees

Designer creates designs

Notice that we don't need:

if employee is Developer:

or:

if employee is Manager:

The same:

employee.work()

works for all objects.

15. Inheritance + Constructor + Methods + Polymorphism

Now let's combine all the concepts.

```
class Employee:

    def __init__(self, name, emp_id, salary):
        self.name = name
        self.emp_id = emp_id
        self.salary = salary
    def display(self):
        print("\nName:", self.name)
        print("Employee ID:", self.emp_id)
```

3DWEBSOFT FOUNDATION

```
        print("Salary:", self.salary)
    def work(self):
        print(self.name, "is working")
class Developer(Employee):
    def work(self):
        print(self.name, "is developing software")
class Manager(Employee):
    def work(self):
        print(self.name, "is managing the team")
class Designer(Employee):
    def work(self):
        print(self.name, "is designing graphics")
developer = Developer("Rahul", 101, 50000)
manager = Manager("Asha", 102, 70000)
designer = Designer("Ravi", 103, 45000)
developer.display()
developer.work()
manager.display()
manager.work()
designer.display()
designer.work()
```

Expected Output

```
Name: Rahul
```

3JWEBSOFT FOUNDATION

Employee ID: 101

Salary: 50000

Rahul is developing software

Name: Asha

Employee ID: 102

Salary: 70000

Asha is managing the team

Name: Ravi

Employee ID: 103

Salary: 45000

Ravi is designing graphics

16. Types of Inheritance

Python supports several forms of inheritance.

1. Single Inheritance

One parent → one child.

Employee

↓

Developer

2. Multilevel Inheritance

SUWEBSOFT FOUNDATION

A → B → C

Employee

↓

Developer

↓

SeniorDeveloper

Example:

```
class Employee:
    pass
class Developer(Employee):
    pass
class SeniorDeveloper(Developer):
    pass
```

3. Multiple Inheritance

One child inherits from multiple parents.

```
SkillA     SkillB
  \       /
   \     /
    Employee
```

Example:

```
class Technical:
    def coding(self):
```

3DWEBSOFT FOUNDATION

```
        print("Coding")
class Communication:
    def speaking(self):
        print("Speaking")
class Employee(Technical, Communication):
    pass
e1 = Employee()
e1.coding()
e1.speaking()
```

4. Hierarchical Inheritance

One parent → multiple children.

```
          Employee
        /    |    \
       ↓     ↓     ↓
  Developer  Manager  Designer
```

This is especially useful for our Employee Management System.

17. Employee Salary Example

We can also customize salary calculations.

```
class Employee:


    def __init__(self, name, salary):
```

3DWEBSOFT FOUNDATION

```
        self.name = name
        self.salary = salary
    def calculate_bonus(self):
        return self.salary * 0.05
class Developer(Employee):
    def calculate_bonus(self):
        return self.salary * 0.10
class Manager(Employee):
    def calculate_bonus(self):
        return self.salary * 0.15
d1 = Developer("Rahul", 50000)
m1 = Manager("Asha", 70000)
print("Developer Bonus:", d1.calculate_bonus())
print("Manager Bonus:", m1.calculate_bonus())
```

Output

```
Developer Bonus: 5000.0
Manager Bonus: 10500.0
```

Here, the same method:

```
calculate_bonus()
```

behaves differently.

This is another example of **polymorphism**.

18. Practical Project – Employee Management System

3DWEBSOFT FOUNDATION

Features

Our program will contain:

- Employee
- Developer
- Manager
- Designer
- Employee details
- Role-specific work
- Salary
- Polymorphism

```
class Employee:


    def __init__(self, name, emp_id, salary):
        self.name = name
        self.emp_id = emp_id
        self.salary = salary


    def display(self):
        print("\nEmployee ID:", self.emp_id)
        print("Name:", self.name)
        print("Salary:", self.salary)


    def work(self):
```

3DWEBSOFT FOUNDATION

```
        print("Employee is working")
    def calculate_bonus(self):
        return self.salary * 0.05
class Developer(Employee):
    def work(self):
        print("Role: Developer")
        print(self.name, "is developing software")

    def calculate_bonus(self):
        return self.salary * 0.10
class Manager(Employee):
    def work(self):
        print("Role: Manager")
        print(self.name, "is managing the team")
    def calculate_bonus(self):
        return self.salary * 0.15
class Designer(Employee):
    def work(self):
        print("Role: Designer")
        print(self.name, "is designing graphics")
    def calculate_bonus(self):
        return self.salary * 0.08
```

3DWEBSOFT FOUNDATION

```
employees = [
    Developer("Rahul", 101, 50000),
    Manager("Asha", 102, 70000),
    Designer("Ravi", 103, 45000)
]
for employee in employees:
    employee.display()
    employee.work()
    print("Bonus:", employee.calculate_bonus())
```

Sample Output

```
Employee ID: 101
Name: Rahul
Salary: 50000
Role: Developer
Rahul is developing software
Bonus: 5000.0

Employee ID: 102
Name: Asha
Salary: 70000
Role: Manager
Asha is managing the team
Bonus: 10500.0
```

3DWEBSOFT FOUNDATION

Employee ID: 103

Name: Ravi

Salary: 45000

Role: Designer

Ravi is designing graphics

Bonus: 3600.0

19. Real-World Applications

Inheritance and polymorphism are widely useful in software development.

Application	Parent Class	Child Classes
Employee System	Employee	Developer, Manager
School	Person	Student, Teacher
Banking	Account	Savings, Current
Vehicles	Vehicle	Car, Bike, Bus
E-commerce	Product	Electronics, Clothing
Hospital	Person	Doctor, Patient

3DWEBSOFT FOUNDATION

20. Common Mistakes

Mistake 1 – Incorrect inheritance

❌

class Developer:

Employee

✅

class Developer(Employee):

Mistake 2 – Forgetting self

❌

def work():

✅

def work(self):

Mistake 3 – Incorrect super()

❌

super.init()

✅

super().__init__()

Mistake 4 – Confusing inheritance and polymorphism

Remember:

Inheritance

↓

Reuses parent functionality

3DWEBSOFT FOUNDATION

Polymorphism

↓

Same method → different behavior

21. Inheritance vs Polymorphism

Feature	**Inheritance**	**Polymorphism**
Main idea	Reuse functionality	Different behavior
Relationship	Parent → Child	Same interface, many forms
Example	Developer(Employee)	developer.work()
Main benefit	Code reuse	Flexible code
Common technique	Class inheritance	Method overriding

22. Classroom Activity

Activity 1 – Vehicle

Create:

class Vehicle:

Create child classes:

class Car(Vehicle):

class Bike(Vehicle):

3DWEBSOFT FOUNDATION

class Bus(Vehicle):

Each class should override:

start()

Expected behavior:

Car → Car engine starts

Bike → Bike engine starts

Bus → Bus engine starts

Activity 2 – School Management

Create:

Person

↓

Student

Teacher

Parent attributes:

- name
- age

Student attributes:

- roll number
- marks

Teacher attributes:

- subject
- salary

3DWEBSOFT FOUNDATION

23. Mini Project Challenge

Employee Management System – Student Task

Create an application containing:

Parent Class

Employee

Attributes:

name

employee_id

salary

Methods:

display()

work()

calculate_bonus()

Child Classes

Developer

Manager

Designer

Each child class should override:

work()

calculate_bonus()

Final requirement

Store all employees in a list:

employees = []

3DWEBSOFT FOUNDATION

Then use a loop to display their details.

24. Quick Revision

INHERITANCE

↓

Parent → Child

↓

Code Reuse

POLYMORPHISM

↓

Same Method

↓

Different Behavior

Important keywords:

class

self

super()

Important concepts:

Parent Class

Child Class

Method Overriding

Constructor

Object

3DWEBSOFT FOUNDATION

Polymorphism

25. MCQs

1. What is inheritance?

A. Deleting a class
B. Getting properties and methods from another class
C. Creating a variable
D. Importing a module

Answer: B

2. Which class is inherited from?

A. Child class
B. Derived class
C. Parent class
D. Object class only

Answer: C

3. Which keyword/function is commonly used to access parent functionality?

A. parent()
B. super()
C. base()
D. upper()

Answer: B

4. What is polymorphism?

A. One class only
B. One method having different behavior

3DWEBSOFT FOUNDATION

C. Removing inheritance
D. Creating files

Answer: B

5. What is method overriding?

A. Deleting a method
B. Child class providing its own version of a parent method
C. Copying a file
D. Creating a variable

Answer: B

6. Which is correct?

class Manager(Employee):

A. Manager inherits Employee
B. Employee inherits Manager
C. Both are unrelated
D. This is invalid

Answer: A

3DWEBSOFT FOUNDATION

26. Practical Assignment

Build an **Employee Management System** with:

Parent

Employee

Children

Developer

Manager

Tester

Requirements

Each employee should have:

- Name
- Employee ID
- Salary

Methods:

display()

work()

calculate_bonus()

Use **polymorphism** to call the same methods for all employees.

Challenge

Add another class:

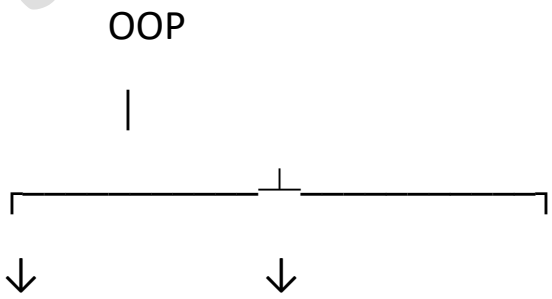
class HR(Employee):

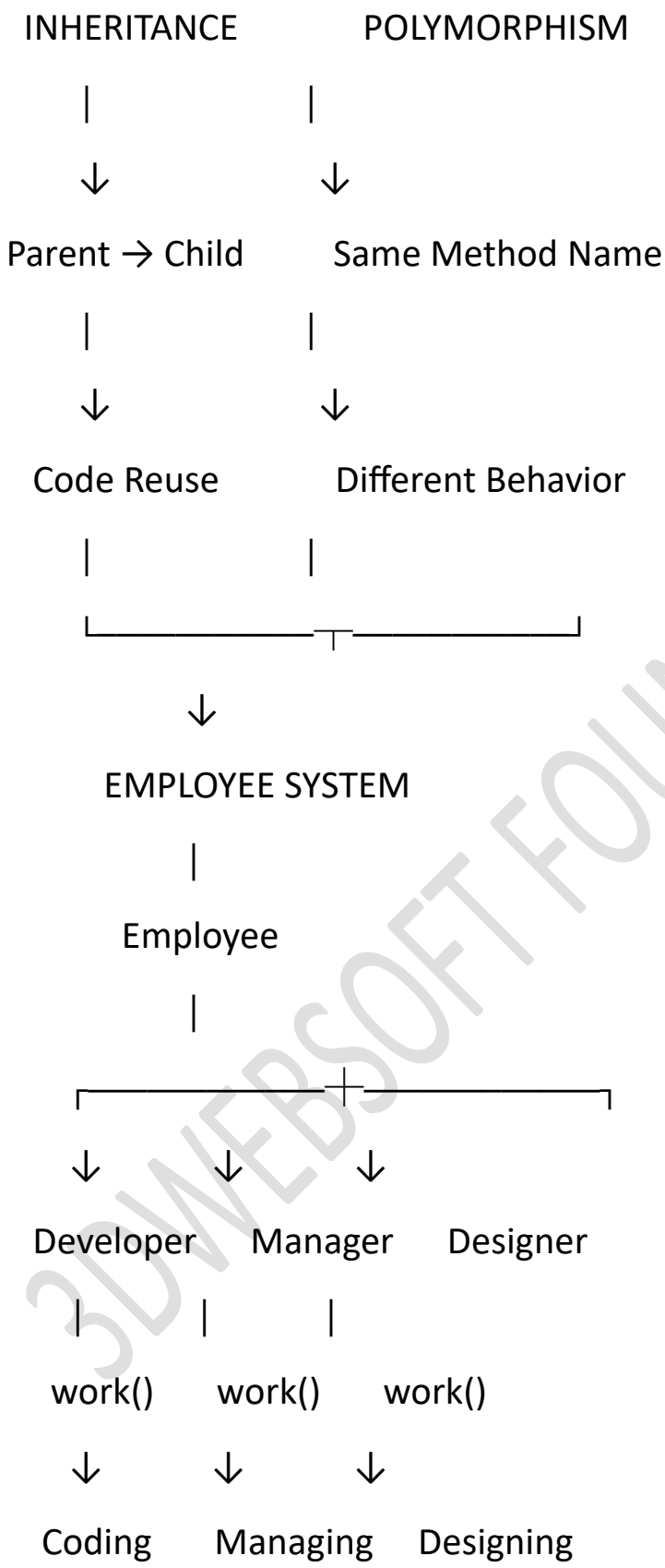
and create its own version of:

work()

3DWEBSOFT FOUNDATION

27. Concept Map





3DWEBSOFT FOUNDATION

Final Takeaway

Students should remember:

Inheritance = “is-a” relationship and code reuse.

Polymorphism = same method/interface with different behavior.

For the Employee Management example:

```
class Developer(Employee):
    def work(self):
        print("Developer is coding")
class Manager(Employee):
    def work(self):
        print("Manager is managing")
```

Both use:

```
employee.work()
```

3DWEBSOFT FOUNDATION